

02-staff-and-access-management

Loaded from 02-staff-and-access-management.md

Staff and Access Management

Audience: Admin operators handling people-level changes: joiners, leavers, recipient list updates, webhook changes, and the admin password.

This file covers the full procedures. The manager-tier equivalent (</docs/waratah/for-managers/04-staff-and-recipients.md>) covers only the safe parts (knowing who is on the list and asking you for changes). This file is the canonical procedure for everything that requires Script Properties or code access.

1. Overview of What Changes When a Person Changes

When a staff member joins, leaves, or changes role at The Waratah, the following touch points may all need updating depending on their access pattern:

Touch point	Lives in	Required for
STAFF_LIST constant in EnhancedTaskManagementWaratah.gs	Code (hard-coded)	Showing in the Task Management Staff Allocated dropdown
WARATAH_EMAIL_RECIPIENTS Script Property	Shift Report project	Receiving nightly PDF email
SLACK_DM_WEBHOOKS Script Property	Both projects	Receiving personal Slack DMs
Slack workspace access	Slack admin (separate)	Reading any channel or DM
Google Drive folder permissions	Drive admin (separate)	Reading archived reports
ESCALATION_EMAIL / ESCALATION_SLACK_WEBHOOK Script Property	Both projects	Receiving BLOCKED-task escalations (typically Evan only)

Not every change needs every touch point. A new floor staff member who never sees the nightly report probably needs only STAFF_LIST . A new manager joining probably needs all six.

2. Adding a New Staff Member (Full Procedure)

Walk through the steps below in order. Skip any step that does not apply.

Step 1: Decide their access level

Before touching any code or Script Properties, decide:

- Do they need to appear in the task assignee dropdown? (Almost always yes for staff who work at the venue.)
- Will they receive the nightly PDF email? (Yes for managers and Evan; no for floor staff.)
- Will they receive personal Slack DMs for the nightly report? (Yes for managers and Evan, opt-in; no for floor staff.)
- Will they receive task-assigned DMs? (Yes if they will be assigned tasks; no otherwise.)
- Will they receive escalations? (Almost always: no, escalations stay with Evan unless deputised.)

Step 2: Add to STAFF_LIST in code

The Staff Allocated dropdown is hard-coded. Edit THE WARATAH/TASK MANAGEMENT SCRIPTS/EnhancedTaskManagementWaratah.gs .

```
const STAFF_LIST = [  
  "Evan",  
  "Cynthia",  
  "Adam",  
  "Jaiden",  
  "Joffy",  
  "Bar Team",  
  "Nick",  
  "Kitchen Team",  
  "All",  
  "Contractor",  
  "FOH Team",  
  "General Management",  
  "Marketing Explicit"  
];
```

Add the new person's name in the named-person section (above "Bar Team"). Save the file.

Deploy via `clasp push` (see [04-deployment-and-clasp.md](#)).

After deployment, in the Task Management spreadsheet run **Task Management >  Admin Tools > Cleanup >  Reapply Dropdowns & Formatting**. This refreshes the dropdown to show the new name. Verify by clicking any Staff Allocated cell and confirming the new name appears.

Step 3: Get their personal Slack DM webhook (if applicable)

If the new person will receive personal DMs, they need to generate their own Slack incoming webhook:

1. The new person opens Slack > Settings & administration > Manage apps > Incoming Webhooks > Add to Slack.
2. They choose to post messages "To a specific channel" and select **their own DM (Slackbot or themselves)**.

3. They click Add Incoming Webhooks Integration. Slack generates a webhook URL.
4. They send the URL to you (the admin).

Treat the URL as a secret. Do not share it beyond the admin role.

Step 4: Add to SLACK_DM_WEBHOOKS Script Property (both projects)

Open the **Shift Report project** Script Properties:

1. Open the shift report spreadsheet, Extensions > Apps Script > Project Settings > Script Properties.
2. Find `SLACK_DM_WEBHOOKS` . Click edit.
3. The value is a JSON object. Add the new entry:

```
{
  "Evan": "https://hooks.slack.com/services/.../existing-ewan",
  ...
  "NewPerson": "https://hooks.slack.com/services/.../new-webhook"
}
```

1. Save.

The Shift Report project copy of `SLACK_DM_WEBHOOKS` is stored for hygiene and symmetry only; the Shift Report runtime does not read it. The Task Management project copy is the one that actually drives task-assigned DMs.

Repeat the same edit in the **Task Management project** (this is the consequential one):

1. Open the Task Management spreadsheet, Extensions > Apps Script > Project Settings > Script Properties.
2. Find `SLACK_DM_WEBHOOKS` . Click edit.
3. Paste the identical JSON.
4. Save.

Skipping the Task Management project means task-assigned DMs will not reach the new person. Skipping the Shift Report project copy is harmless to nightly behaviour but leaves the two projects out of sync, which makes future debugging harder.

Step 5: Add to WARATAH_EMAIL_RECIPIENTS (if applicable)

Open the Shift Report project Script Properties. Find `WARATAH_EMAIL_RECIPIENTS` . The value is a JSON object mapping email to display name:

```
{
  "ewan@pollenhospitality.com": "Evan",
  ...
  "newperson@pollenhospitality.com": "NewPerson"
}
```

Add the new entry, save.

This property is **only** in the Shift Report project. The Task Management project does not have an equivalent (task notifications go via Slack DM, not email).

Step 6: Verify

The Send TEST shift report flow does not exercise `SLACK_DM_WEBHOOKS`. To verify the new person's personal DM, you must trigger a Task Management dropdown change instead:

1. Open the Task Management spreadsheet.
2. Create or assign a test task to the new person (set Staff Allocated to their name and Status to TO DO).
3. Confirm the new person receives a Slack DM about the assignment.
4. To verify the email recipient list separately, send a TEST shift report from the shift report spreadsheet (**Waratah Tools > Daily Reports > Export & Email (TEST to me)**). Note: the TEST send delivers a single PDF email to the configured test recipient (Evan by default), not to the full `WARATAH_EMAIL_RECIPIENTS` distribution. To verify a new addition to the LIVE distribution, perform a controlled LIVE send and confirm receipt.

If anything fails, see [03-advanced-troubleshooting.md](#).

Step 7: Tell managers

Once technical setup is complete, message the manager team: "Joffy is now in the dropdown and on the recipient lists. Tasks assigned to Joffy will DM her in Slack."

3. Removing a Departing Staff Member (Full Procedure)

Inverse of adding, but with one extra critical step: reassign their open tasks before removal.

Step 1: Reassign open tasks

In the Task Management spreadsheet, filter by Staff Allocated = the departing person. Reassign each open task (NEW, TO DO, IN PROGRESS, TO DISCUSS, BLOCKED, DEFERRED) to another person or CANCEL it. Recurring tasks owned by the departing person need their template re-pointed.

This step must complete BEFORE removing them from `STAFF_LIST`. Otherwise the Reapply Dropdowns step in Step 3 will leave orphaned tasks pointing at a name that no longer exists in the dropdown.

Step 2: Remove from `WARATAH_EMAIL_RECIPIENTS`

Shift Report project Script Properties. Find `WARATAH_EMAIL_RECIPIENTS`, remove their email entry, save.

Step 3: Remove from SLACK_DM_WEBHOOKS (both projects)

Both projects. Find the entry, delete it, save in each.

Step 4: Remove from STAFF_LIST in code

Edit EnhancedTaskManagementWaratah.gs . Remove the name from the list. Save and clasp push.

Step 5: Reapply dropdowns

Run **Task Management >  Admin Tools > Cleanup >  Reapply Dropdowns & Formatting.**

This refreshes the Task Management dropdown so the removed name disappears from new task entry.

Step 6: Slack workspace and Drive cleanup

These are separate admin tasks (Slack workspace settings, Drive sharing) and live outside this system.

Step 7: Verify

Send a TEST shift report. Confirm the removed person:

- Does not appear in the email Bcc/To list (check the TEST email or Logger output).
- Does not receive a Slack DM (they should not see anything in Slack).

4. Changing Email Recipients

If you are not adding or removing a staff member but want to change who receives the nightly PDF (for example, adding the bookkeeper temporarily):

1. Shift Report project Script Properties > WARATAH_EMAIL_RECIPIENTS .
2. Edit the JSON object. Add or remove entries.
3. Save.
4. Send a TEST report to verify the new recipient list.

Email recipients are independent of STAFF_LIST . Someone can receive emails without being in the task assignee dropdown (and vice versa).

5. Changing Slack Webhooks

Slack webhooks may need to change in three situations:

1. **The Slack channel is renamed or archived.** Webhooks become invalid. You need to regenerate the webhook in Slack and update the property.
2. **A person's personal webhook expires or is regenerated.** They give you the new URL.
3. **The Slack workspace itself changes.** All webhooks need regeneration.

Procedure (one webhook at a time)

For each webhook that needs changing:

1. Get the new webhook URL (from Slack admin or the recipient).
2. Open the relevant project's Script Properties.
3. Find the property (`WARATAH_SLACK_WEBHOOK_LIVE` , `WARATAH_SLACK_WEBHOOK_TEST` , `SLACK_MANAGERS_CHANNEL_WEBHOOK` , `ESCALATION_SLACK_WEBHOOK` , or the right key inside `SLACK_DM_WEBHOOKS`).
4. Paste the new URL. Save.
5. If the property exists in both projects (see [01-configuration-reference.md](#) Section 8), repeat in the other project.
6. Send a TEST shift report. Confirm the new webhook posts to the expected destination.

If multiple webhooks need updating at once (workspace move), do them all in one session and run a single TEST send at the end.

6. Changing the Admin Password

The `MENU_PASSWORD` property gates about 20 admin menu items in the shift report spreadsheet and another set in the Task Management spreadsheet. Both projects share the same password.

When to change

- After a staff member who knew the password leaves.
- On a scheduled rotation (every 6 months is a reasonable cadence).
- If you suspect the password has leaked.

Procedure

1. Pick a new password. Write it down somewhere outside the system (password manager).
2. Shift Report project Script Properties > `MENU_PASSWORD` . Set the new value. Save.
3. Task Management project Script Properties > `MENU_PASSWORD` . Set the same new value. Save.
4. From either spreadsheet, try a password-gated menu action (for example **Waratah Tools > Admin Tools > Open Analytics Viewer**). Enter the new password. Confirm it accepts.
5. Try the same action with the OLD password. Confirm it rejects.

If the two projects' passwords drift, you will get the confusing situation where the password works in one spreadsheet's menus but not the other.

Password recovery

The password is stored only in Script Properties. If the admin forgets it, recovery requires direct edit access to Script Properties, which is available to anyone with edit access to the Apps Script project (script owner, or anyone the owner has granted Edit). There is no separate "reset password" flow.

If you have lost edit access to the Apps Script project entirely, contact Google Workspace admin (workspace owner can grant access via the file's sharing settings).

7. Admin Access Policies

Who needs admin access

Currently only Evan has admin access. The role specifically covers:

- Editing Script Properties on either Apps Script project.
- Pushing code via clasp.
- Installing or removing time-based triggers.
- Knowing the admin password (which gates menu actions like Rollover, Backfill, Dashboard Rebuild, Send TEST/Weekly Summary).
- Owning the script's Drive permissions (Apps Script project ownership, archive folder permissions).

Granting another person admin access

If you need to add a second admin (holiday cover, succession planning):

1. Share both Apps Script projects with the new admin's Google account, with Edit permission.
2. Share the Archive Root Drive folder with the new admin (Edit).
3. Tell them the current admin password.
4. Tell them about this documentation directory.

Revoking admin access

When a former admin leaves:

1. Remove their Google account from the Apps Script project sharing (both projects).
2. Remove from Drive folder sharing.
3. Rotate the admin password (Section 6 above).
4. If they had personal Slack webhooks, remove them (Section 3 above).
5. If they were the `ESCALATION_EMAIL` target, change the property to the new admin's email.

8. The Two-Project Dual-Update Rule (Reminder)

Six properties are written into both projects' Script Properties stores:

- `MENU_PASSWORD` (functionally dual-read; both projects use it)
- `TASK_MANAGEMENT_SPREADSHEET_ID` (functionally dual-read; both projects use it)
- `SLACK_MANAGERS_CHANNEL_WEBHOOK` (hygiene only in SR; functionally read only by TM)
- `SLACK_DM_WEBHOOKS` (hygiene only in SR; functionally read only by TM)

- `ESCALATION_EMAIL` (hygiene only in SR; functionally read only by TM)
- `ESCALATION_SLACK_WEBHOOK` (hygiene only in SR; functionally read only by TM)

For the first two, dual update is required. For the other four, the Task Management copy is the consequential one; the Shift Report copy is hygiene. Best practice is still to keep them in sync. When editing one, immediately switch tabs to the other project's Script Properties and apply the same change.

9. Logging Changes

After any people-level change (add, remove, recipient list edit, password rotation), add a one-line note to the change log at `docs/waratah/_archive/CONFIG_CHANGE_LOG.md`:

```
2026-05-17: Added Joffy to STAFF_LIST. Added Joffy + Jaiden to SLACK_DM_WEBHOOKS in both projects. Reapplied dropdowns. TEST send verified.
```

This is a discipline, not enforced by the system. Future debugging depends on it.